



VELS





INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)
(Deemed to be University Estd. u/s 3 of the UGC Act, 1956)
PALLAVARAM, THALAMBUR, PERIYAPALAYAM, THIRUVANMIYUR - CHENNAI

CAMPAIGN PERFORMANCE PREDICTION AND BUDGET OPTIMIZATION SYSTEM

MINI PROJECT REPORT

Submitted by

Harishwaran K (24619142)
Mageshwaran H (24619144)
Keerthivarman M (24619145)

Under the guidance of
Mrs. V. P. Kanchana

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

VELS INSTITUTE OF SCIENCE, TECHNOLOGY & ADVANCED STUDIES (VISTAS)

CHENNAI - 600 117, INDIA
2026

ISBN No: 978-93-344-7370-4

BONAFIDE CERTIFICATE

This is to certify that the mini-project report entitled **"CAMPAIGN PERFORMANCE PREDICTION AND BUDGET OPTIMIZATION SYSTEM"** is a record of the work presented by **HARISHWARAN K (24619142), MAGESHWARAN H (24619144), KEERTHIVARMAN M (24619145)** under the guidance of **Mrs. V P KANCHANA**, Professor,VELS Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, during the academic year 2026–2027.

The project report is submitted in partial fulfilment of the academic requirements prescribed by the institution. The details and results included in this report are based on the information and implementation summary provided by the project team and should be verified against the final project artefacts before submission.

SIGNATURE OF THE HOD	SIGNATURE OF THE SUPERVIOR
Dr. K. Ulaga Priya M.E., Ph.D	Mrs. V P Kanchana
Associate Professor & Head	Professor
Of Department AI&ML	VISTAS
School of Engineering	Chennai
VISTAS	
Chennai	

Submitted for the Project Work and Viva - Voce examination held

On…… at Vels Institute of Science, Technology and Advanced Studies (VISTAS),

Chennai – 600 117.

INTERNAL EXAMINER EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We express our sincere gratitude to VELS Institute of Science, Technology & Advanced Studies (VISTAS), Chennai, for providing the academic environment for this mini project.

We thank our project guide, **Mrs. V. P. Kanchana**, for guidance and support. We also thank the faculty members of the Department of Artificial Intelligence and Machine Learning.

We thank our teammates and everyone who contributed feedback, testing assistance, and encouragement during the development of the virtual try-on prototype.

We acknowledge the researchers whose work on contextual bandits and advertising budget optimization informed our literature review. We also acknowledge the supplied project materials and deployed applications used to describe the proposed system and its visible interface.

Harishwaran K | Mageshwaran H | Keerthivarman M

ABSTRACT

Digital advertising plays a significant role in modern marketing, enabling businesses to reach target audiences through multiple advertising channels. However, campaign performance varies depending on campaign characteristics, channel selection, and budget allocation, making it challenging to utilize advertising resources effectively. This paper proposes a system that integrates machine-learning-based campaign performance prediction with constrained advertising budget optimization to support data-driven marketing decisions. Using historical campaign data, the system applies supervised learning techniques to analyze campaign characteristics and predict expected performance across different campaigns and advertising channels. The system employs a Gradient Boosting Regressor to predict campaign revenue based on relevant campaign characteristics. Based on the predicted outcomes, an optimization layer determines a suitable allocation of the available marketing budget while satisfying predefined budget constraints. The system also incorporates a what-if analysis feature that enables users to modify budget allocations or campaign parameters and examine their potential impact on predicted performance and resource distribution. A web-based dashboard presents prediction results, channel recommendations, optimized budget allocations, and simulation outcomes in an accessible format. By integrating predictive analytics, channel recommendation, constrained optimization, and scenario analysis, the proposed system aims to improve advertising resource utilization, minimize inefficient spending, and support informed digital advertising campaign planning.

TEAM MEMBERS

1. Harishwaran K (24619142)

2. Mageshwaran H (24619144)

3. Keerthivarman M (24619145)

PROJECT GUIDE

Mrs. V. P. Kanchana

TABLE OF CONTENTS

Section	Page
Bonafide Certificate	2
Declaration and Acknowledgement	3
Abstract	4
Chapter 1 - Introduction	7
Chapter 2 - Literature Review	9
Chapter 3 - System Analysis and Design	11
Chapter 4 - Implementation and Results	15
Project Interface Screenshots	17
Chapter 5 - Conclusion and Future Work	26
References	28
Appendix A - Requirements and Observation Notes	29
Chapter 6 - Code Level Implementation	30
Chapter 7 - Evaluation and Operational Design	50
Appendix B - Selected Source Excerpts	59
Appendix C - API and Data Contract Notes	63
Appendix D - Glossary and Traceability	65

LIST OF FIGURES

Figure 3.1 Proposed system architecture 11

Figure 4.1 Executive dashboard 17

Figure 4.2 Campaign setup and targeting configuration 18

Figure 4.3 Budget allocation engine 19

Figure 4.4 Cross-platform analytics 20

Figure 4.5 A/B split comparison demo 21

Figure 4.6 Audience segmentation demo 22

Figure 4.7 What-if budget simulator 23

Figure 4.8 Executive report view 24

Figure 4.9 Ad creative workspace 25

LIST OF TABLES

Table 2.1 Related-work comparison 10

Table 3.1 Functional requirements 11

Table 3.2 India-market budget constraints shown in the UI 13

Table 4.1 Runtime observations from the deployed interface 16

Table 4.2 Recommended model-evaluation metrics 16

ABBREVIATIONS

API - Application Programming Interface; A/B - two-variant comparison; CPA - Cost Per Acquisition; CPC - Cost Per Click; CTR - Click-Through Rate; GB - Gradient Boosting; KPI - Key Performance Indicator; MAE - Mean Absolute Error; ML - Machine Learning; RMSE - Root Mean Squared Error; R-squared (R2) - coefficient of determination; SLSQP - Sequential Least Squares Programming; UI - User Interface.

CHAPTER 1 - INTRODUCTION

1.1 Project Overview

Digital advertising gives organizations access to search, social, display, and other media channels. A marketing team must decide how to divide a limited budget across campaigns whose returns depend on audience, creative, timing, channel, and spend. A static allocation or a rule based only on past totals can miss diminishing returns: spending more on one channel may produce progressively smaller gains while another channel still has room to grow.

This project proposes a decision-support system that joins two tasks. A supervised machine-learning model estimates campaign revenue from historical campaign characteristics. A constrained optimizer then uses the predicted response to recommend an allocation that respects a total budget and per-channel bounds. A web dashboard makes the inputs, recommendation, and scenario workflow visible. The revised abstract identifies a Gradient Boosting Regressor for revenue prediction and proposes what-if analysis as part of the system [4].

1.2 Problem Statement

Campaign managers need to compare alternative allocations before money is committed. Manual spreadsheets can calculate totals but do not learn a prediction function from historical data or automatically solve a constrained allocation problem. The project addresses the following question: given campaign context, historical performance information, and a fixed budget, how can the system estimate expected revenue and find a feasible allocation across supported advertising platforms?

1.3 Objectives

- Prepare historical campaign attributes for supervised prediction, with a clearly defined target and a leakage-safe feature set.
- Estimate expected campaign revenue using the Gradient Boosting Regressor specified in the final abstract.
- Use predicted response curves in a constrained budget-allocation step.
- Expose campaign setup, channel comparison, and what-if inputs in a web dashboard.
- Report actual model and optimizer results only after the backend and evaluation data are available.

1.4 Scope and Users

The intended users are marketing analysts and campaign planners who need a transparent recommendation for a proposed campaign budget. The deployed interface presents a campaign setup flow, a budget optimizer, analytics, an A/B comparison view, audience insights, a what-if simulator, a report view, and an ad creative workspace [7]. The core research scope is prediction plus allocation. The interface also contains demo areas; those are described as demonstrations rather than treated as evidence of live customer-level analytics.

1.5 Applications and Expected Benefits

- Compare predicted campaign responses across advertising channels before a budget is committed.
- Check that platform allocations sum to the chosen budget and remain within stated minimum and maximum values.
- Explore alternative budget envelopes with a what-if workflow.
- Present campaign context and model output in a single reviewable dashboard.

The expected benefit is better structured decision-making. The project does not guarantee that an allocation will improve revenue in production; that conclusion would require measured, out-of-sample evaluation and, ultimately, controlled campaign evidence.

1.6 Challenges

Challenges include limited or unrepresentative historical data, differences between training and deployment markets, delayed conversions, attribution error, changing auction prices, and missing campaign context. Feature timing is critical: information that becomes known only after a campaign runs cannot be used as an input for a genuine pre-campaign prediction. A mathematically feasible allocation can still be poor if the fitted revenue curves are inaccurate or if an objective omits risk and business rules.

1.7 Report Organization

Chapter 2 reviews advertising allocation research and the project's research gap. Chapter 3 describes requirements, architecture, prediction, and optimization. Chapter 4 documents the deployed interface and verified runtime observations. Chapter 5 summarizes the work and recommends further validation.

CHAPTER 2 - LITERATURE REVIEW

2.1 Advertising Allocation as a Learning Problem

Advertising returns are uncertain and may change with audience, season, creative, channel, and spend. Research approaches range from offline prediction and mathematical optimization to sequential learning methods that observe outcomes and update later decisions. The literature supplied for this project emphasizes a key distinction: predicting performance is not the same operation as allocating a budget. A useful decision-support pipeline must connect its estimate to an explicit constraint-aware decision rule [5, 6].

2.2 Selected Base Paper

Han and Gabor's AdKDD 2020 paper studies budget selection when campaign payouts are uncertain. Their contextual-bandit system predicts payouts from context and historical performance, extrapolates payout distributions to budgets not directly observed, and applies Thompson Sampling to balance exploration and exploitation [1]. The authors report a 22 +/- 10 percent improvement in mean Cost Per Acquisition over a previous allocation strategy. That reported result belongs to their production system at Lyft; it is not a result of this student project [1].

The paper is relevant because it links prediction to a budget decision. Its production-scale contextual-bandit loop is outside the current project's stated scope. The student prototype instead describes supervised revenue prediction followed by constrained optimization. It does not claim to reproduce online bandit learning, sequential experimentation, or automatic live bidding.

2.3 Other Related Work

Nuara, Trovo, Gatti, and Restelli formulate joint bid and daily-budget decisions for pay-per-click advertising as a combinatorial semi-bandit problem. Their work considers online decision-making across campaigns, with the bid and daily budget selected together [2]. Gigli and Stella study parametric bandits for performance marketing, including multi-ad-group campaigns and bootstrapped Thompson sampling [3]. These works demonstrate the breadth of the research area; they also show that production advertising decisions may include bidding, delayed outcomes, exploration, and online updates beyond a static allocation prototype.

2.4 Comparison and Research Gap

Aspect	Research literature	This project
Primary decision	Sequential budget and/or bid selection under uncertain returns	Predict campaign revenue, then optimize a fixed budget
Learning setup	Contextual or combinatorial bandit methods	Supervised Gradient Boosting model in the final abstract
Decision step	Explore/exploit or joint bid-budget optimization	SLSQP allocation with explicit total and platform bounds in the UI
Evidence	Empirical or production results reported by each paper	No verified project metrics available in the supplied materials
User interface	Not the central contribution of the cited methods	Dashboard, what-if, analytics, and demo pages

The project does not claim that advertising optimization is novel. Its practical gap is integration: present a student-accessible workflow that connects a revenue predictor to constrained allocation, exposes inputs and assumptions, and provides a what-if interface. The project reference document explicitly separates the research foundation from the proposed implementation [5].

2.5 Literature Review Summary

The selected base paper motivates the use of predictions in budget decisions. More recent work illustrates sequential and multi-channel alternatives [2, 3]. The project's contribution is a simplified, inspectable prototype. Its value must be judged on data quality, prediction error, feasibility, and whether an allocation improves outcomes when compared with a suitable baseline. Until those measurements are completed, claims remain design goals rather than demonstrated impact.

CHAPTER 3 - SYSTEM ANALYSIS AND DESIGN

3.1 Existing and Proposed System

A manual planning process often combines separate campaign reports, spreadsheet calculations, and marketer judgement. This approach can be hard to reproduce and may not satisfy platform-specific spending limits. The proposed system organizes a campaign definition, performance estimate, allocation, and scenario view into one application. The visible interface labels its backend as FastAPI ML and its optimization endpoint as POST /optimize [7].

3.2 Functional Requirements

ID	Requirement	Verification evidence
FR1	Enter campaign name, industry, brand, market, objective, duration, target, and budget	Campaign Setup page exposes these fields
FR2	Enforce total budget and platform bounds	UI states equality and per-platform limits
FR3	Generate a revenue prediction and channel allocation	Requires an available optimizer backend
FR4	Compare channel efficiency and allocations	Analytics page provides a comparison area
FR5	Run a hypothetical budget scenario	What-if page requires a baseline and backend
FR6	Present campaign summary for review or printing	Reports page presents an executive report view

3.3 System Architecture

PROJECT PIPELINE
1. Historical campaign data and campaign context
2. Data validation and feature preparation
3. Gradient Boosting revenue prediction
4. Platform response curves and constrained SLSQP optimization
5. Allocation, analytics, and what-if scenario output
6. Web dashboard and report presentation

Figure 3.1: Proposed prediction-to-allocation system architecture. Conceptual architecture based on the project proposal and visible application design.

The architecture separates prediction from allocation. Historical campaign records are cleaned and transformed into model features. The predictor estimates expected revenue for supported campaign context. An optimization layer evaluates channel-level response against the available budget and bounds. The user interface collects inputs and displays outputs. The supplied conference draft proposed this pipeline and cloud access; the deployed page currently exposes a FastAPI ML interface and labels Gradient Boosting and SciPy SLSQP components [6, 7].

The application's connection to the backend is an important boundary. The updated deployment completed the optimizer request during the latest inspection. This verifies the sample workflow, not model quality or production behavior.

3.4 Data Flow and Processing

- Input: campaign metadata and historical records. Proposed historical fields include spend, impressions, clicks, CTR, conversions, CPC, CPA, channel, and campaign type [6].
- Validation: check missing values, units, duplicate records, currency, date ranges, and category labels.
- Feature preparation: encode categories and derive only values that are available at prediction time.
- Prediction: apply the selected Gradient Boosting Regressor to estimate the target revenue.
- Optimization: evaluate platform response and solve for an allocation subject to total-budget and platform constraints.
- Presentation: display the allocation, predicted revenue, ratios, constraints, and scenario comparison.

Leakage control is essential. Spend, clicks, conversions, CPA, or CTR measured after a campaign may reveal the outcome being predicted. They should be included only when their timestamps establish that they are known at decision time, or be excluded from the pre-campaign prediction set. The final dataset, target column, and feature timing were not included with the supplied files, so this report does not claim a validated training design.

3.5 Prediction and Optimization Formulation

Let x denote campaign context and let f(x) estimate expected revenue. For m supported channels, let b_i be the budget assigned to channel i and R_i(b_i, x) its predicted response. The allocation can be written as maximizing the sum of expected channel responses subject to a fixed budget and lower and upper bounds:

Objective	**maximize $\Sigma_i R_i(b_i, x)$**
Budget equality	$\Sigma_i b_i = B$
Feasible bounds	$l_i \leq b_i \leq u_i$ for each supported channel

The deployed budget page describes a channel-specific logarithmic response curve and a Sequential Least Squares Programming (SLSQP) minimizer. A representative form displayed in the interface is $R_i(b_i) = \alpha_i \log(1 + \beta_i b_i) + \gamma_i$, which captures diminishing marginal returns as spend increases [7]. The response parameters are displayed by the UI; their derivation and calibration data were not included in the supplied materials. They are therefore implementation settings, not independently validated estimates.

3.6 Market Constraints Visible in the Interface

Market	Supported platforms	Minimum and maximum	Total budget
India	Google Ads, Meta Ads	Google: ₹5,000-₹25,000; Meta: ₹5,000-₹15,000	₹10,000-₹40,000

For the India scope the UI excludes TikTok Ads and says the platform allocations must sum exactly to the chosen total. These limits are configuration displayed by the application, not a general regulatory statement. The selected sample configuration is ₹25,000 across Google and Meta, with 30 days, a ₹75,000 target revenue goal, SaaS industry, Zoho brand, and Search Engine Marketing objective [7]. In the latest sample run, Google received ₹14,928 and Meta received ₹10,072; both amounts fall within the displayed platform bounds and sum to the configured ₹25,000. A feasible allocation requires the total budget to remain between the sum of the platform minimums and the sum of their maximums.

The total budget is a hard equality constraint: Google allocation plus Meta allocation must equal the campaign budget. Platform bounds prevent the optimizer from placing an infeasible amount on either channel. For India, Google must receive ₹5,000 to ₹25,000 and Meta ₹5,000 to ₹15,000, so a valid total must be between ₹10,000 and ₹40,000. The latest ₹25,000 sample allocation assigns ₹14,928 to Google and ₹10,072 to Meta. Both amounts fall within their respective ranges and sum exactly to the selected total. This 60/40 split depends on the campaign configuration, market, model response curves, and selected budget; it is not a fixed recommendation for every campaign. If a proposed total falls outside the combined feasible range, the system should report infeasibility rather than silently violate a platform bound. The revenue estimate is a model output, not observed sales, and requires validation against campaign data.

3.7 Main Modules and Requirements

Module	Responsibility	Current evidence
Campaign setup	Collect campaign metadata and budget	Rendered form and default values
Prediction and optimizer	Estimate revenue and allocate budget	UI describes Gradient Boosting and SLSQP; latest sample run returned ₹87,240
Analytics	Show allocation, revenue, ratios, and channel comparison	Page loaded; values awaited optimizer
What-if simulator	Compare baseline and alternative budget	Page requires baseline and live POST request
A/B testing and audience	Illustrate comparison and audience views	Explicit demo labels; audience data marked synthetic
Reports and ad studio	Present executive summary and ad preview	Pre-filled report and copy workspace visible

Non-functional requirements include clear constraint messages, consistent currency handling, reproducible model versions, responsive page layout, validation of user inputs, and transparent labeling of demo data. A production deployment should protect campaign data, limit access to uploaded datasets, log model versions and input dates, and make stale predictions visible. Since campaign performance and budget details may be commercially sensitive, cloud deployment should include access control and data-retention policies.

3.8 Software and Hardware Context

The deployed interface identifies a FastAPI ML service. The budget page names SciPy SLSQP. The available project material does not provide a complete dependency lock file, exact server version, training hardware, or dataset size. The website was inspected at the public Vercel deployment on 29 October 2026. The updated deployment completed an optimizer run during inspection on 30 October 2026, returning a sample predicted revenue of ₹87,240 for the ₹25,000 budget [7].

CHAPTER 4 - IMPLEMENTATION AND RESULTS

4.1 Implementation Overview

The revised abstract defines a supervised revenue-prediction and constrained-allocation system [4]. The code archive provides concrete evidence for a FastAPI backend, a separate model-training pipeline, a response-curve optimizer, and a React frontend. In the archived implementation the API endpoint uses the response-curve optimizer directly; the saved Gradient Boosting model is not imported by that endpoint. The deployed interface provides complementary screen-level evidence, while the archive describes a code snapshot whose integration settings may differ from the live deployment [7].

The default configuration visible in the application is Omnichannel Growth Q3, Zoho, SaaS, India, Search Engine Marketing, 30 days, target revenue ₹75,000, and total budget ₹25,000. The India envelope shown is ₹10,000 to ₹40,000, split between Google Ads and Meta Ads with stated platform bounds. These are the application's pre-populated sample values, not observed results from a real campaign.

4.2 Prediction and Optimizer Implementation

The archive contains a Gradient Boosting training script and a separate platform response-curve optimizer. In the supplied snapshot, the FastAPI endpoint calls the response-curve optimizer; it does not load the saved Gradient Boosting model. The displayed deployment run previously recorded in this report returned ₹87,240 for a ₹25,000 sample, but that UI observation cannot be attributed to the archived model code. The archive's model metrics report MAE ₹17,134.34, RMSE ₹27,743.90, and R-squared 0.2198 on a random 80/20 split (1,440 training and 360 test records). Those metrics describe the separately trained model artifact; they do not validate the endpoint's fixed response-curve optimizer.

4.3 Interface Evidence

Figures 4.1-4.9 capture the deployed interface before the latest successful optimizer run. They document the page layout and default workflow; the live run output is summarized in Section 4.2. The A/B page describes a uniform baseline and an optimized variant but notes that the optimized allocation depends on live optimizer output. The audience page explicitly labels its profiles as synthetic demo data. The reports page shows a pending model run. The ad generator contains pre-filled SaaS copy and a preview; its presence alone does not establish that an external generative model was called.

4.4 Runtime Observations and Evaluation Plan

Check	Observation	Status
Dashboard load	Campaign overview rendered; current sample result displayed	Observed
Campaign setup	Inputs and India constraints rendered with sample defaults	Observed
Budget optimizer request	₹87,240 predicted revenue; 60/40 Google-Meta allocation; bounds respected	Completed
Analytics	Page rendered; optimizer output behavior not rechecked after the latest run	Observed
What-if workflow	No baseline campaign optimization; simulation requires live request	Not completed

Audience and A/B pages	Demo labels visible; audience profiles marked synthetic	Demo interface only

4.5 Metrics Required for Final Evaluation

Measure	Definition / purpose	Use
MAE	Mean absolute difference between actual and predicted revenue	Interpretable average error
RMSE	Square root of mean squared prediction error	Penalizes larger errors
R-squared	Proportion of target variance explained on held-out data	Compare predictive fit
Constraint residual	Absolute difference between total budget and sum of allocations	Verify equality constraint
Scenario delta	Change in predicted revenue or allocation between budgets	Assess what-if sensitivity

These metrics are an evaluation plan, not reported outcomes. A credible final experiment should state dataset source and size, feature definitions, time-aware train/test split, baseline model, selected model, hyperparameters, and all measured results. Allocation quality should be compared with simple baselines such as equal split and historical allocation, while reporting feasibility and sensitivity to budget bounds.

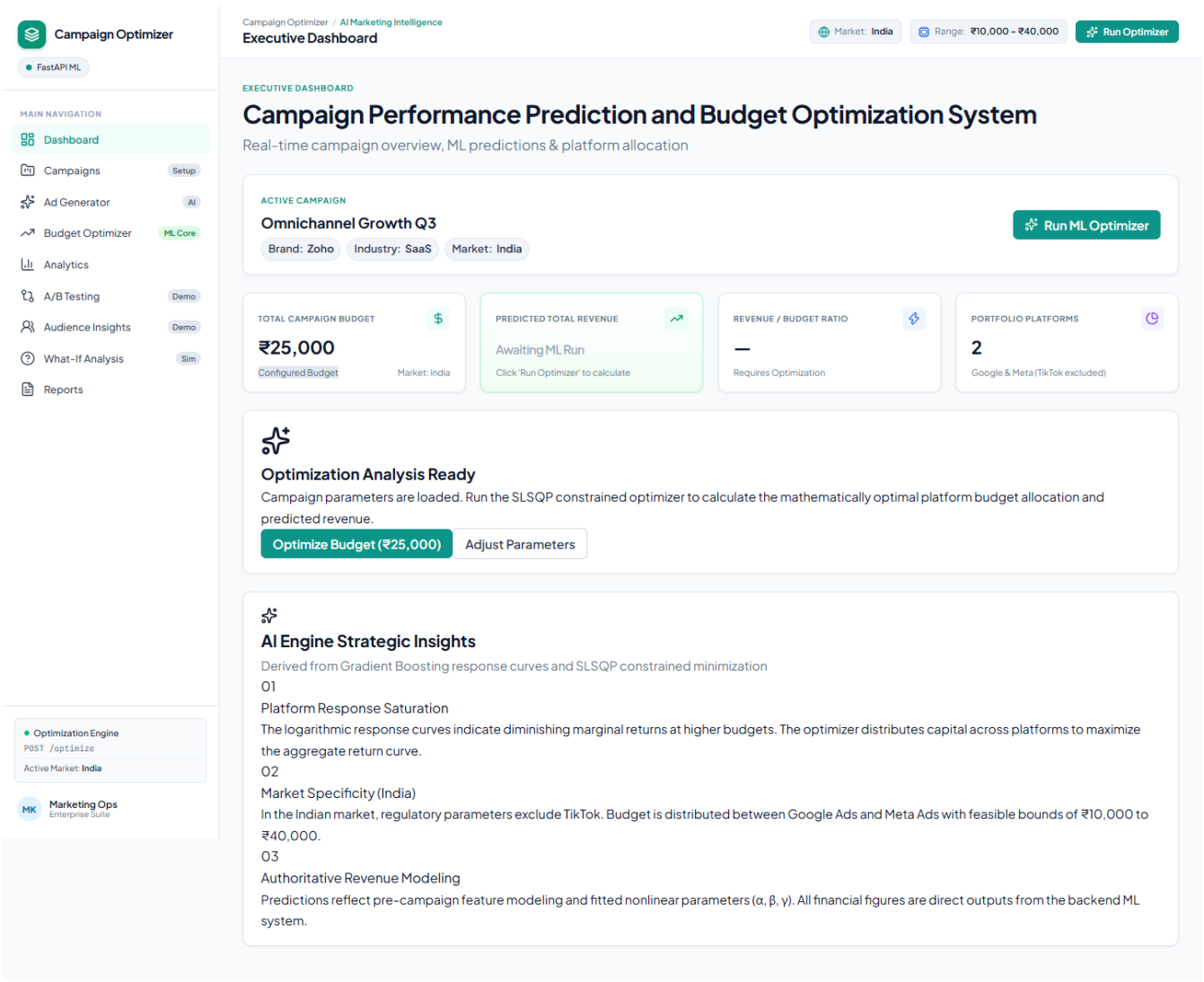


Figure 4.1: Executive dashboard showing the sample campaign, ₹25,000 configured budget, two-platform portfolio, and pending prediction fields.

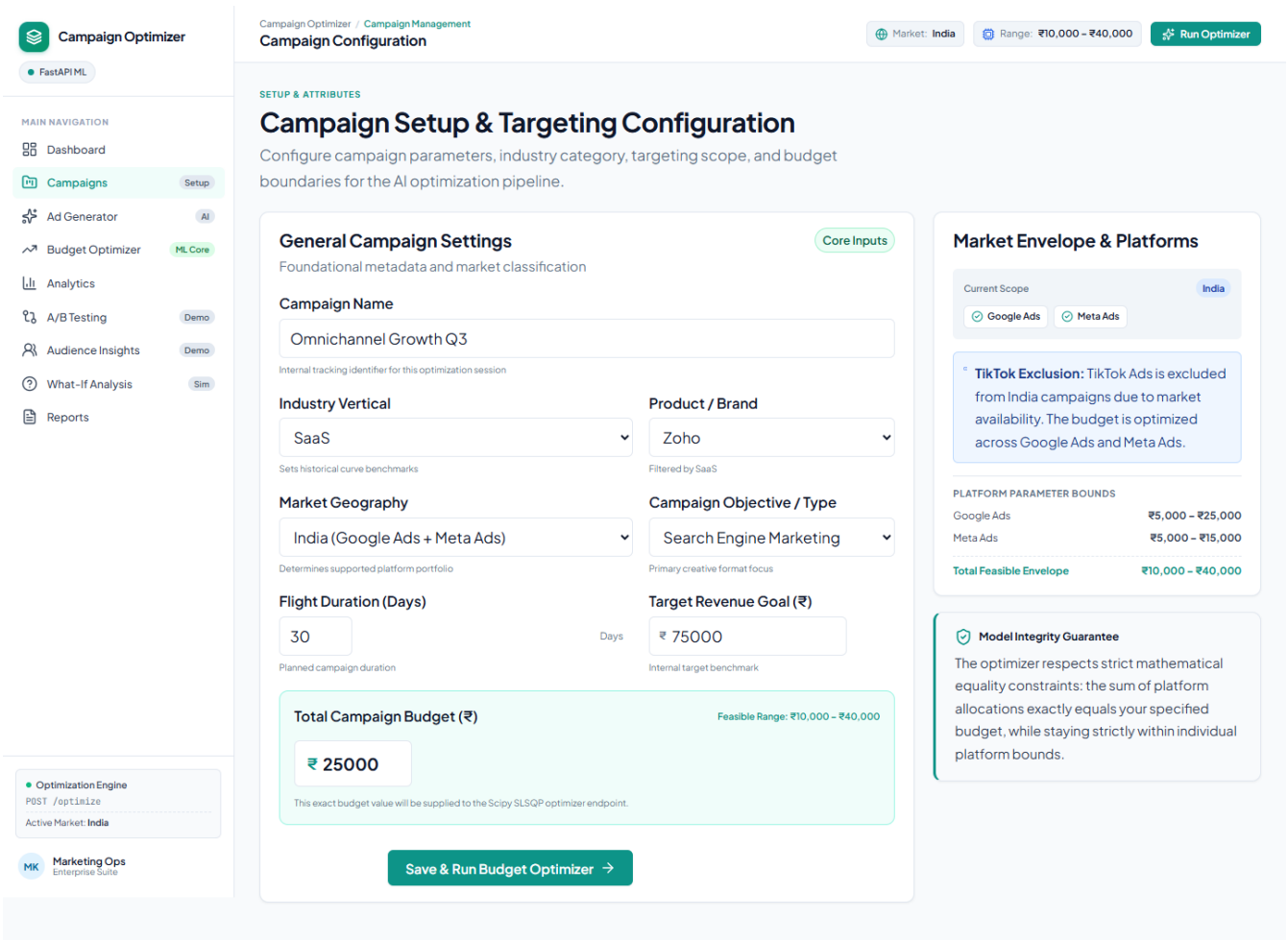


Figure 4.2: Campaign setup page showing campaign inputs, the India market envelope, platform bounds, and TikTok exclusion.

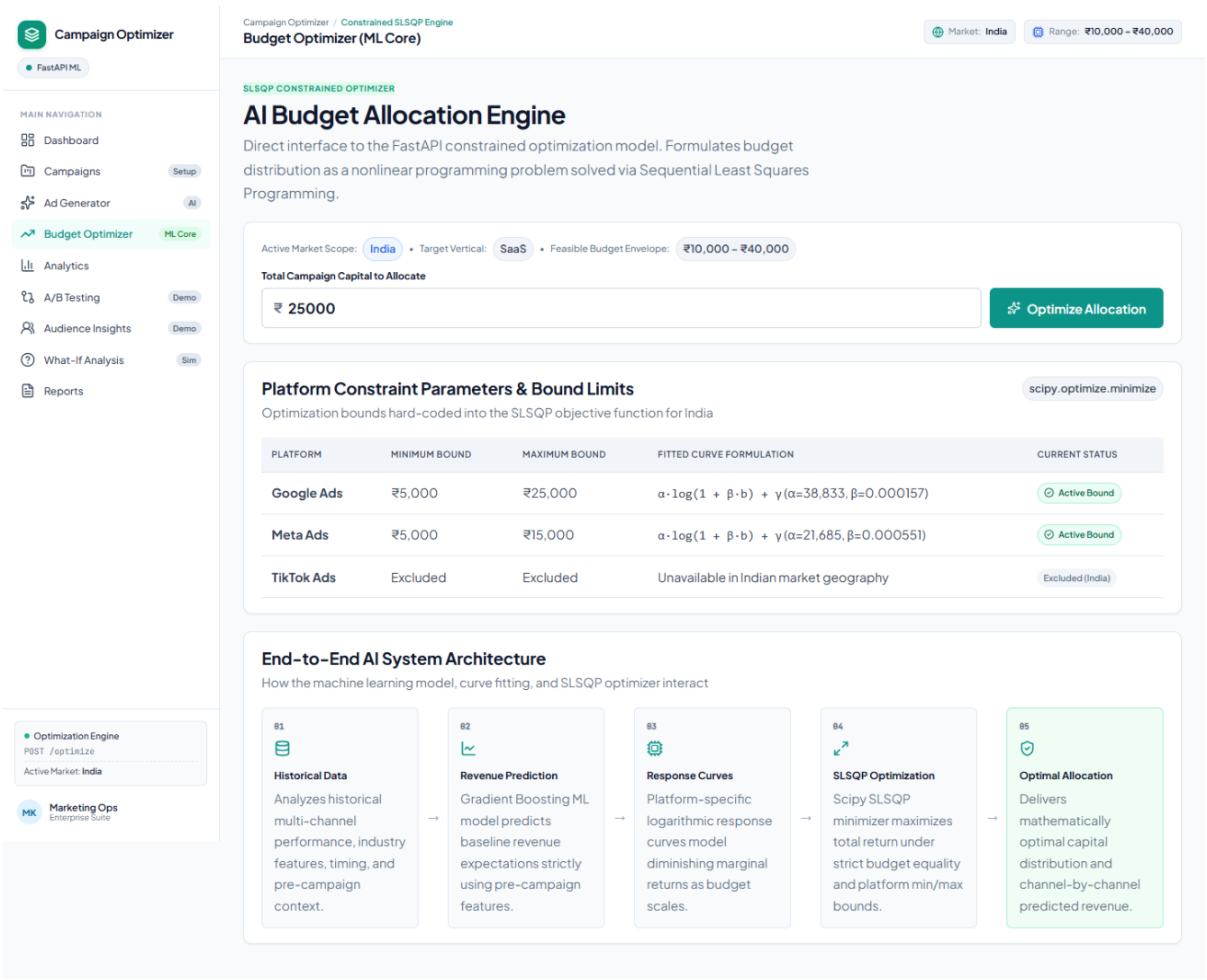


Figure 4.3: Budget optimizer page describing the SLSQP workflow, response curves, platform bounds, and ₹25,000 sample budget.

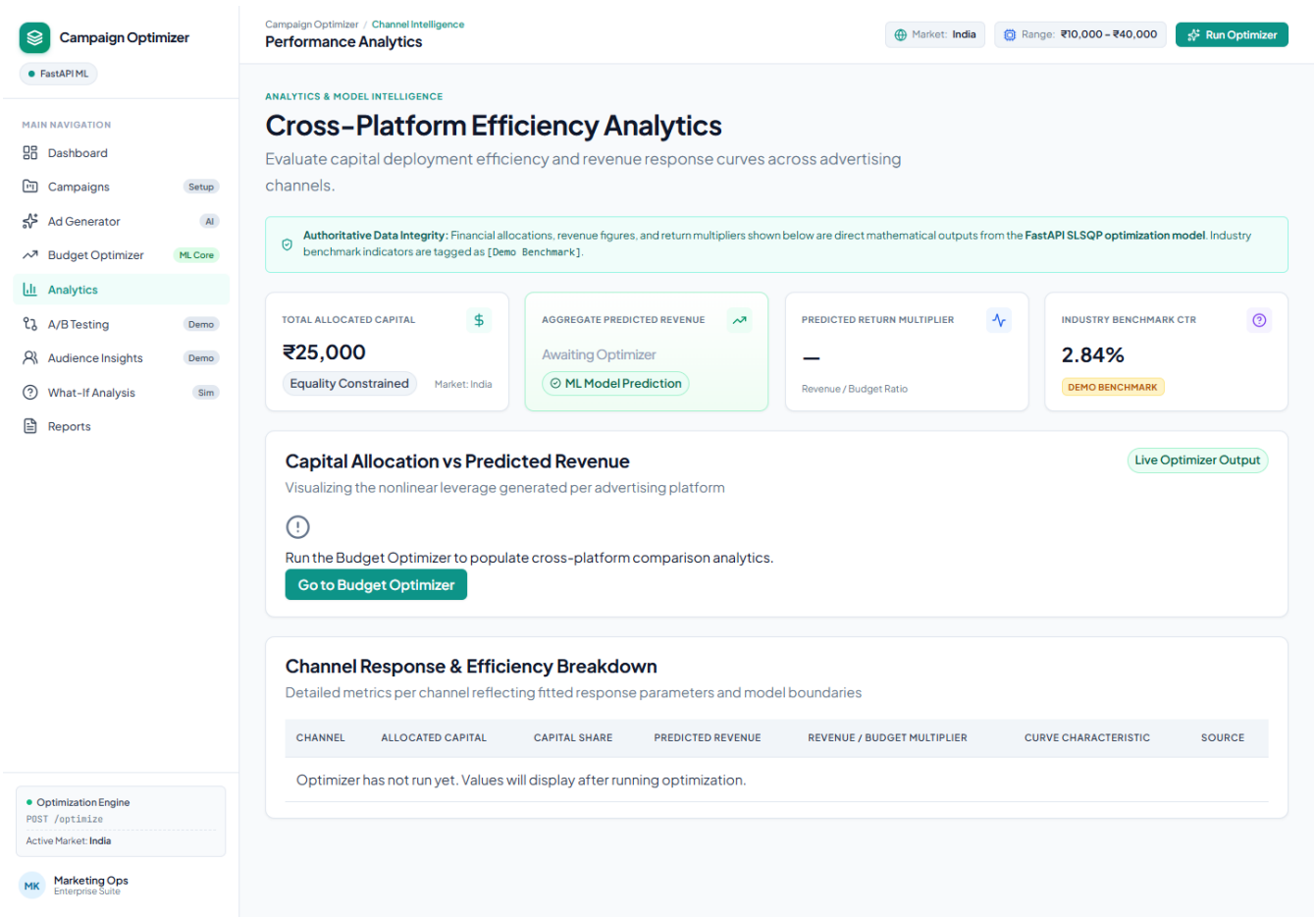


Figure 4.4: Analytics page with pending prediction output and an explicitly labeled demo benchmark CTR.

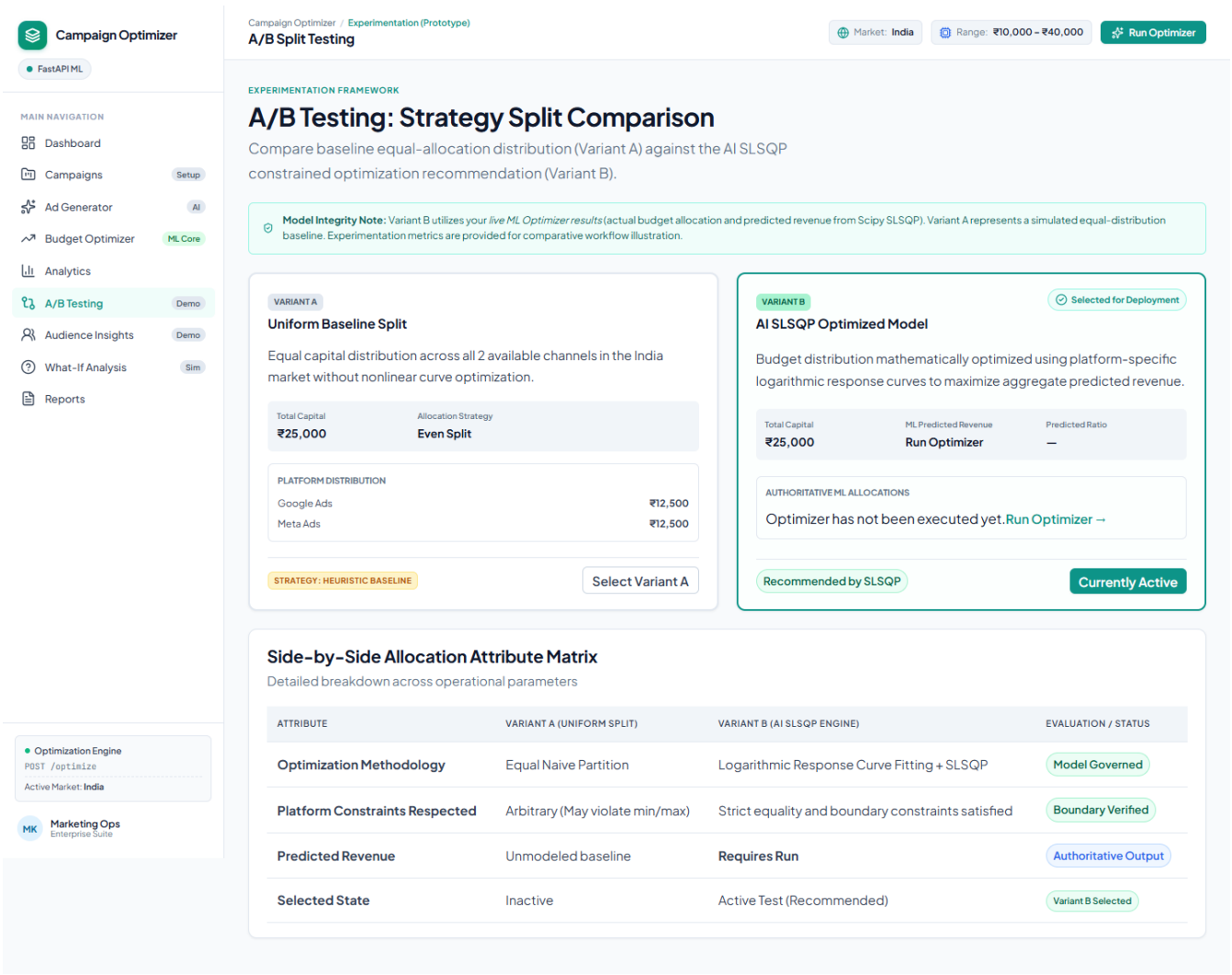


Figure 4.5: A/B split comparison page. The optimized variant depends on live optimizer results; its metrics are presented as workflow illustration.

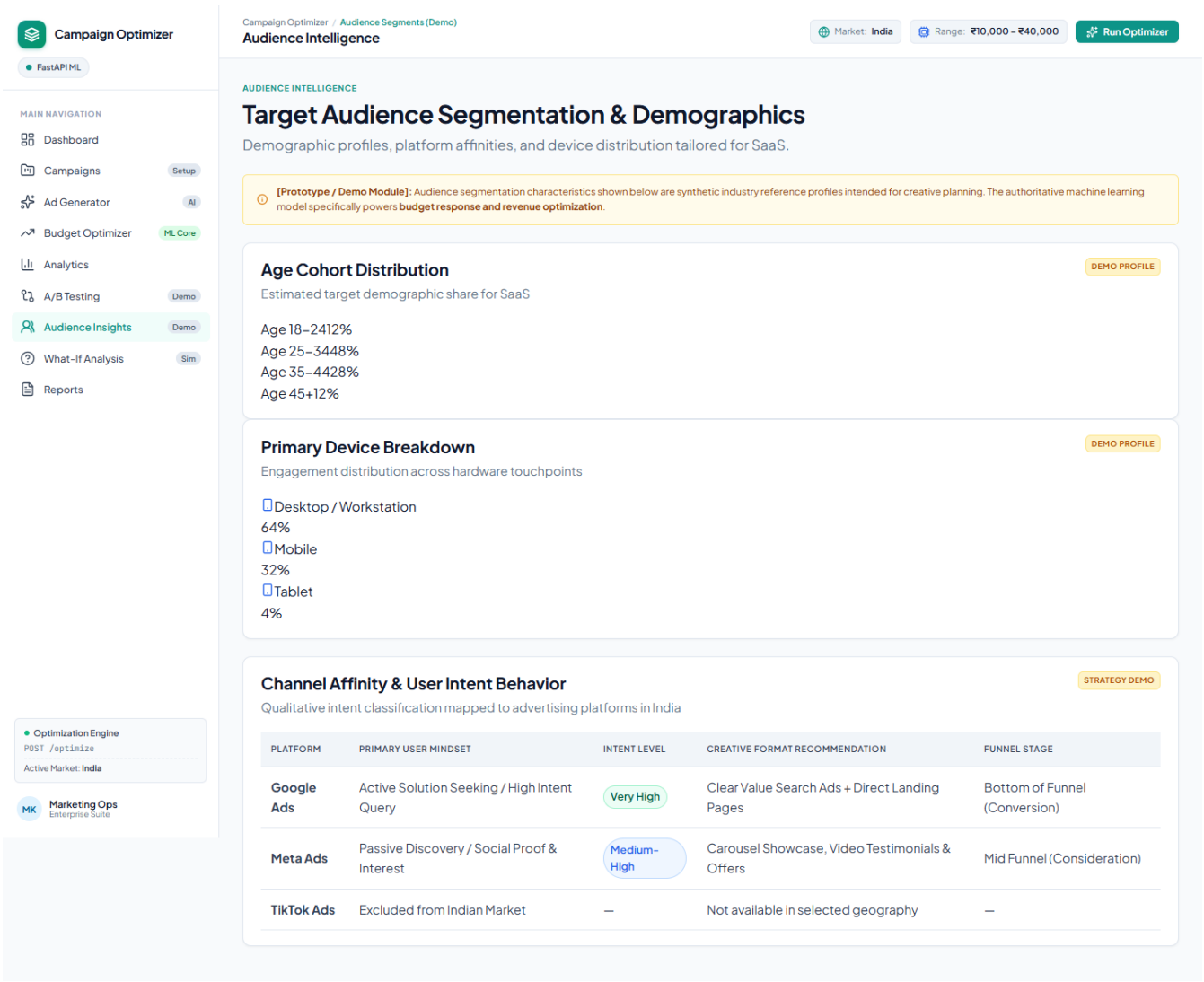


Figure 4.6: Audience segmentation page. The visible notice identifies its demographic profiles as synthetic industry reference data.

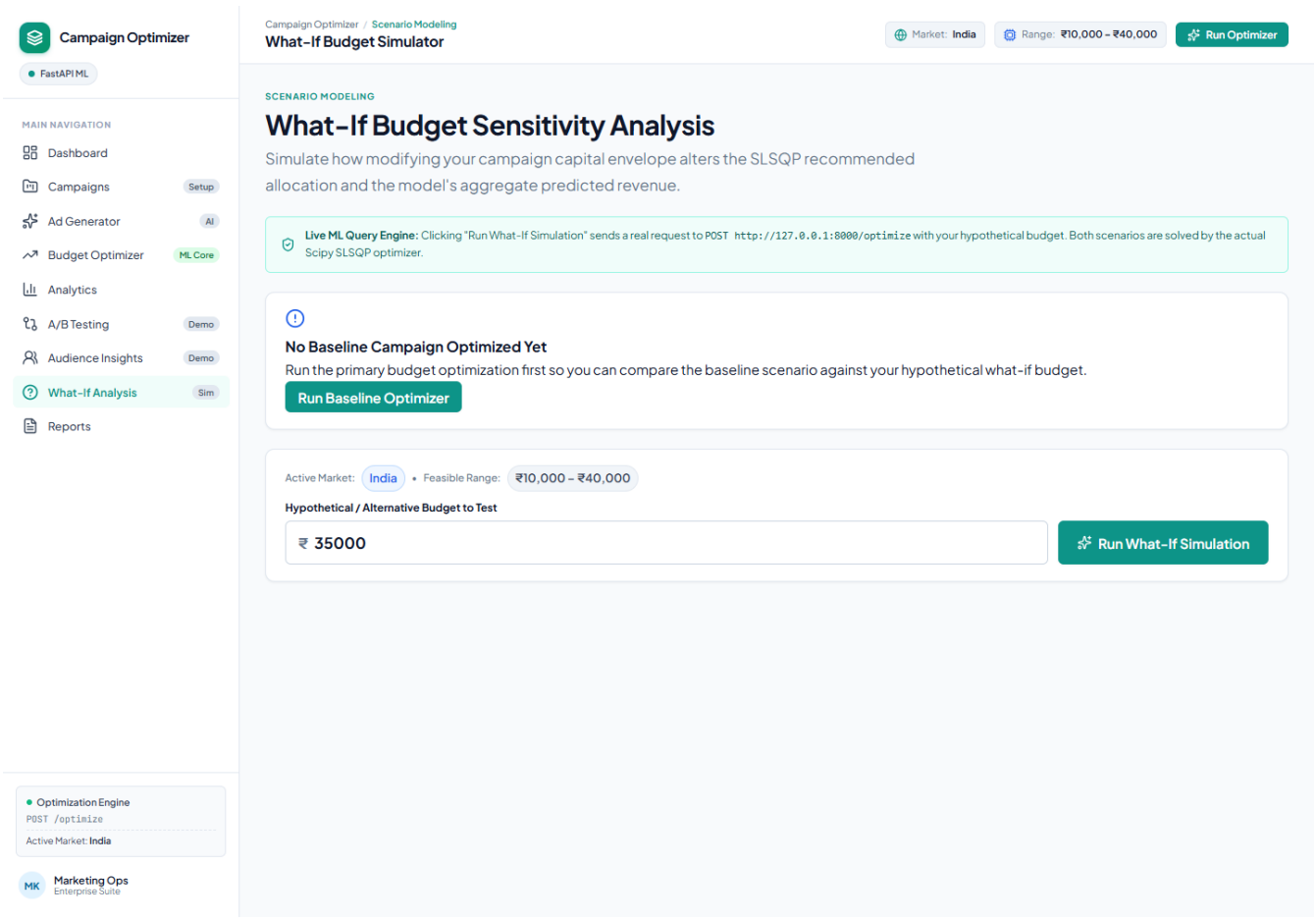


Figure 4.7: What-if budget simulator showing the ₹35,000 sample scenario and the requirement to run a baseline first.

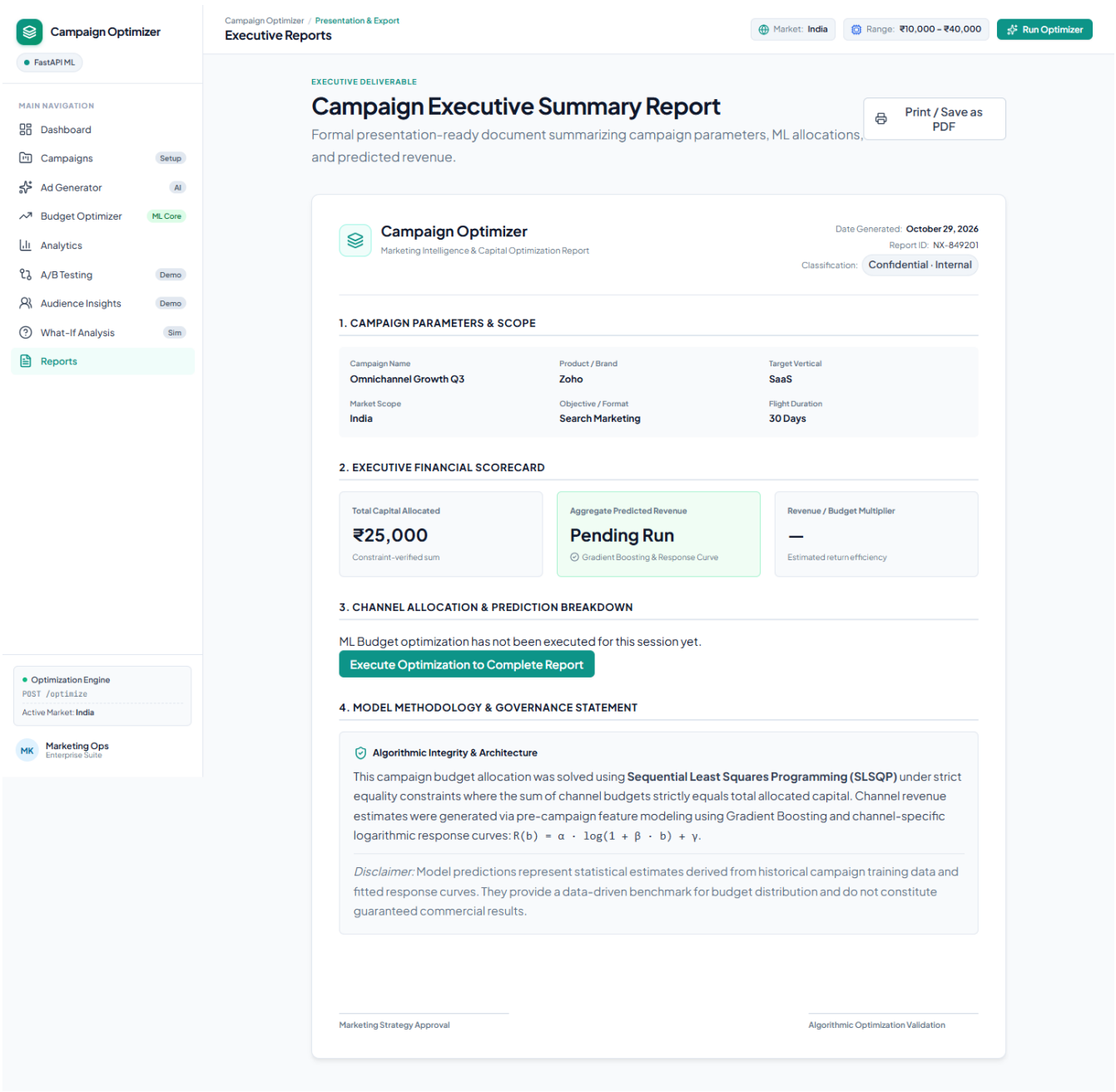


Figure 4.8: Executive report view with a pending revenue prediction and an action to execute optimization.

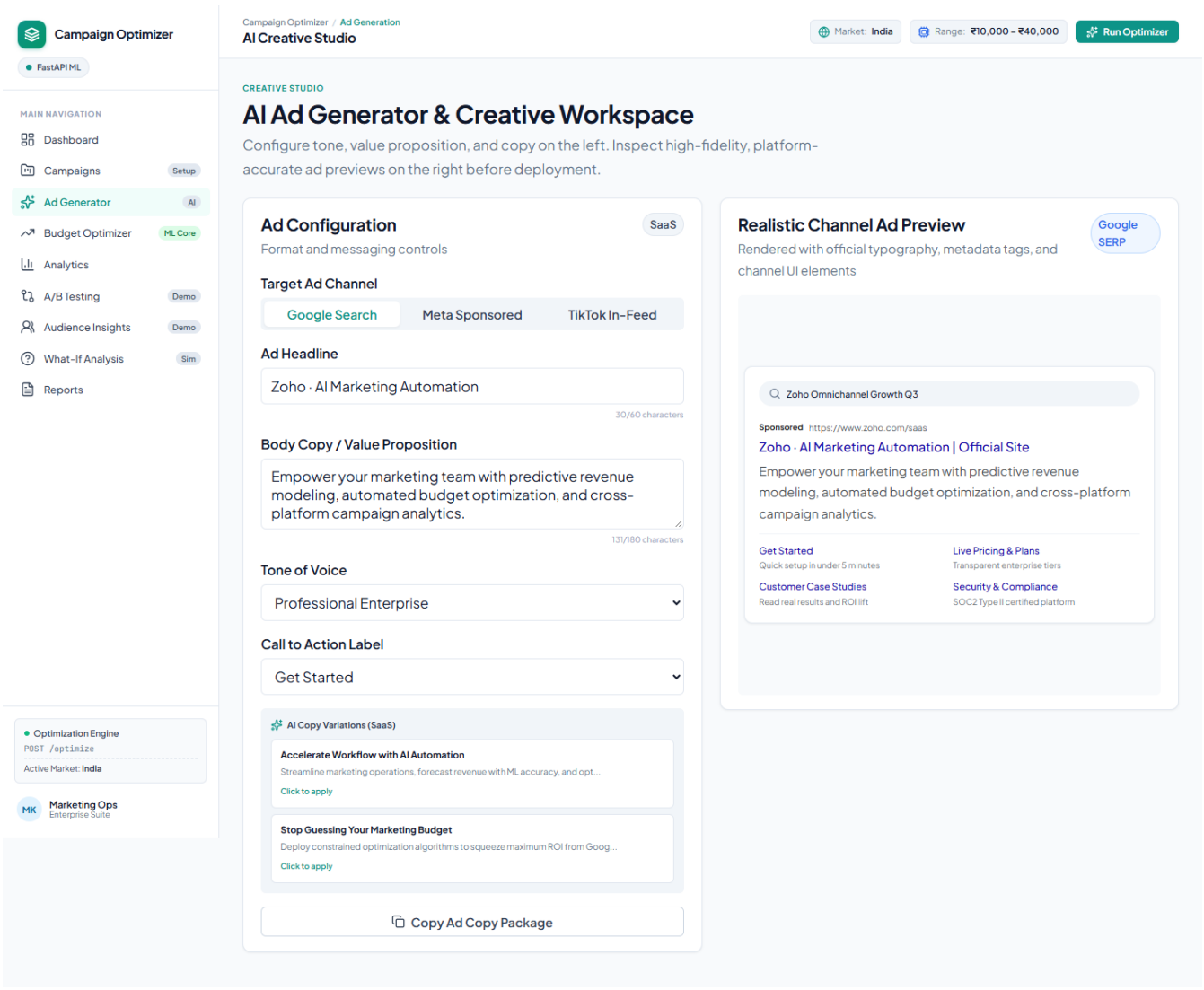


Figure 4.9: Ad creative workspace with pre-filled SaaS copy, channel controls, and a Google Search preview.

CHAPTER 5 - CONCLUSION AND FUTURE WORK

5.1 Conclusion

The project brings campaign revenue prediction and constrained budget allocation together as a marketing decision-support workflow. Its final abstract names the Gradient Boosting Regressor and what-if analysis. The deployed interface shows campaign setup, India-specific platform constraints, an optimizer page describing SLSQP, analytics, demo comparisons, and report presentation. The supplied base paper provides research context for using predicted campaign outcomes to support allocation decisions [1, 4-7].

The updated deployment returned a live sample prediction and budget allocation during inspection. The report makes no claim about model accuracy, revenue uplift, or the optimizer's real-world benefit; those require documented training data and measured evaluation. The interface defaults document what the application is configured to present; measured performance must come from a completed backend run and a reproducible dataset evaluation.

5.2 Project Contribution

The practical contribution is the integration of a campaign form, predictive and optimization workflow, market constraints, scenario interface, and report-oriented dashboard in one accessible prototype. This contribution is appropriately framed as a student implementation of established research ideas, rather than a new advertising-allocation algorithm.

5.3 Future Work

- Record the deployed FastAPI endpoint version and the exact request and response schema for reproducibility.
- Document the historical dataset, record count, date coverage, target variable, feature timing, and data-cleaning decisions.
- Train and evaluate the Gradient Boosting model with a time-aware held-out set; report MAE, RMSE, and R-squared, with a simple baseline.
- Validate response-curve fitting and units; inspect whether channel revenue curves are monotonic and whether parameter values are calibrated on held-out campaigns.

- Test optimizer feasibility at lower, upper, and boundary budgets, including budgets where platform bounds make an allocation infeasible.
- Run controlled what-if scenarios and report allocation and predicted-revenue changes without presenting them as causal effects.
- Separate live model outputs from demo benchmarks and synthetic audience profiles in the UI and in exported reports.
- Add automated tests for input validation, constraints, error handling, API timeouts, and report generation.
- Document data retention, campaign confidentiality, model versioning, and cloud deployment configuration.

5.4 Final Summary

The application presents a clear path from campaign context to allocation review. Its next evidence milestone is not a larger claim; it is a repeatable run with documented data and verifiable outputs. Once those are available, this report can be updated with measured model results, allocation tables, and completed what-if comparisons.

REFERENCES

[1] B. Han and J. Gabor, “Contextual Bandits for Advertising Budget Allocation,” AdKDD 2020. Available: https://www.adkdd.org/papers/contextual-bandits-for-advertising-budget-allocation/2020

[2] A. Nuara, F. Trovo, N. Gatti, and M. Restelli, “Online Joint Bid/Daily Budget Optimization of Internet Advertising Campaigns,” Artificial Intelligence, vol. 305, article 103663, 2022. doi: 10.1016/j.artint.2022.103663.

[3] M. Gigli and F. Stella, “Multi-armed bandits for performance marketing,” International Journal of Data Science and Analytics, vol. 20, pp. 151-165, 2025. doi: 10.1007/s41060-023-00493-7.

[4] H. K. Harishwaran, H. Mageshwaran, and M. Keerthivarman, “Campaign Performance Prediction and Budget Optimization System,” revised project abstract, 2026.

[5] Project team, “Base Paper Study & Project Correlation: Cloud-Based AI Campaign Performance Prediction & Budget Optimization System,” project reference document, 2026.

[6] H. K. Harishwaran, H. Mageshwaran, and M. Keerthivarman, “Cloud-Based AI Campaign Performance Prediction and Budget Optimization System,” conference paper draft, 2026.

[7] Campaign Performance Prediction and Budget Optimization System, deployed web application. Available: https://campaign-budget-optimizer-cznq.vercel.app/ (accessed 30 October 2026).

APPENDIX A - REQUIREMENTS AND OBSERVATION NOTES

A.1 Sample campaign configuration visible in the deployed UI

Field	Displayed value	Interpretation
Campaign	Omnichannel Growth Q3	Interface default
Brand / industry	Zoho / SaaS	Interface default
Market / objective	India / Search Engine Marketing	Interface default
Duration / target	30 days / ₹75,000	Target is not predicted revenue
Total budget	₹25,000	Within displayed feasible envelope
Platforms	Google Ads and Meta Ads	TikTok excluded in India
Alternative scenario	₹35,000	What-if input shown; simulation not completed

A.2 Interpretation of interface evidence

A rendered page verifies that a screen and its displayed labels were available. It does not by itself verify server-side execution, model quality, data provenance, causal lift, or production readiness. Screens marked Demo or Prototype in the interface are treated accordingly. The updated deployment returned a sample prediction of ₹87,240 and a Google/Meta allocation of ₹14,928/₹10,072; these displayed outputs do not establish model quality or real-world lift.

A.3 Reproducibility and acceptance checks

A complete evaluation should record the dataset version, prediction target, feature timestamps, train/test split, model version, prediction metrics, optimizer constraints, request and response payloads, and budget scenario. The same input should produce a documented result under the same model and configuration. Demo benchmarks and synthetic audience profiles should remain clearly separated from live campaign outputs.

CHAPTER 6 - CODE LEVEL IMPLEMENTATION

6.1 Source Snapshot and Evidence Scope

The supplied archive contains project code alongside a bundled Python virtual environment, frontend dependencies, generated files, and datasets. This chapter describes only first-party code needed to understand the project: the FastAPI endpoint, preprocessing script, training pipeline, response-curve optimizer, and React request path. Third-party packages, compiled files, caches, and binary model internals are excluded. The archive is treated as a dated source snapshot; code details are not automatically attributed to the currently deployed website.

The central implementation distinction is important. The API imports optimize_budget from models.budget_response_optimizer. That module uses fixed platform-specific logarithmic response curves and SciPy SLSQP. The archive separately includes models/train_model.py, which trains and saves a Gradient Boosting pipeline. The API module does not load that saved pipeline. Therefore the report describes a trained ML artifact and an API optimizer as two code paths, not as a verified single inference pipeline.

6.2 Backend API Entry Point

The backend uses FastAPI and Pydantic. The root endpoint is a service check. The optimization endpoint accepts a total budget and market, invokes the response-curve optimizer, and returns the allocation, per-platform predicted revenues, and their sum. The excerpt below preserves the essential route logic while omitting explanatory comments.

```
class BudgetRequest(BaseModel):
    total_budget: float = Field(ge=10000, le=60000)
    market: str = "Global"

@app.post("/optimize")
def optimize(request: BudgetRequest):
    if request.market not in ["India", "Global"]:
        raise ValueError("Market must be either India or Global")
    allocation, predictions, total_revenue = optimize_budget(
        request.total_budget, market=request.market
    )
    return {
        "total_budget": request.total_budget,
        "market": request.market,
        "allocation": allocation,
        "predicted_revenue": predictions,
        "total_predicted_revenue": total_revenue
    }
```

The route's explicit response dictionary is useful for the frontend because it separates the campaign total, market, platform allocation, and platform revenue estimate. The request schema sets a broad numeric range, while the optimizer enforces narrower feasible totals for each market.

6.3 Request Validation and Feasible Bounds

Two layers of budget checks appear in the supplied source. Pydantic rejects totals below ₹10,000 or above ₹60,000. The optimizer then derives the market-specific feasible range by adding the selected platforms' minimum and maximum budgets. India selects Google Ads and Meta Ads, resulting in a feasible total from ₹10,000 through ₹40,000. Global selects Google, Meta, and TikTok, resulting in a feasible range from ₹15,000 through ₹60,000.

Market	Platforms	Minimum total	Maximum total
India	Google + Meta	₹10,000	₹40,000
Global	Google + Meta + TikTok	₹15,000	₹60,000

The frontend also validates its displayed market-specific limits before sending a request. Server-side checks remain necessary because a client can be bypassed. In the current API source, an invalid market raises ValueError; a production API should map this case to a clear HTTP 4xx response rather than rely on an unhandled server error.

6.4 API Input and Output Contract

A successful request uses JSON with total_budget as a number and market as either India or Global. The code's output includes allocation as a platform-to-budget mapping, predicted_revenue as a platform-to-revenue mapping, and total_predicted_revenue as a scalar. No campaign name, industry, duration, brand, revenue goal, historical data, or trained-model version is accepted by this endpoint.

```
POST                                                                                    /optimize
Content-Type:                                                                       application/json

{"total_budget":                    25000,                    "market":                    "India"}

Response                                                                                 fields:
  total_budget
  market
  allocation:                                        {platform:                             budget}
  predicted_revenue:                                    {platform:                         revenue}
  total_predicted_revenue
```

This compact API supports a budget-allocation workflow but does not, by itself, personalize results using the extra fields in the campaign form. The report therefore distinguishes fields displayed by the UI from fields used by the archived backend.

6.5 Market Routing and Platform Selection

The optimizer selects a platform set from the market value. For India it uses Google Ads and Meta Ads. For Global it includes TikTok Ads as a third option. The available platforms are then used consistently to select the minimums, maximums, response parameters, and result labels. This makes market selection an algorithmic input: it changes both the feasible allocation space and the objective being optimized.

```
if market == "India":
    platforms = ["Google Ads", "Meta Ads"]
else:
    platforms = ["Google Ads", "Meta Ads", "TikTok Ads"]
minimums = np.array([PARAMS[p]["min_budget"] for p in platforms])
maximums = np.array([PARAMS[p]["max_budget"] for p in platforms])
```

The API itself restricts market to the two supported labels before calling the optimizer. The UI also resets previous results when the market changes, preventing a result calculated for one platform set from remaining visible under another selection.

6.6 Platform Revenue Response Function

The archived optimizer defines an expected revenue function for each platform using three fixed parameters. The mathematical form is alpha multiplied by log(1 + beta times budget), plus gamma. The logarithm introduces diminishing marginal increments as budget rises. Different alpha, beta, and gamma values produce different response curves.

```
def revenue_curve(budget, alpha, beta, gamma):
    return alpha * np.log1p(beta * budget) + gamma

def predicted_revenue(platform, budget):
    p = PARAMS[platform]
    return revenue_curve(budget, p["alpha"], p["beta"], p["gamma"])
```

These are programmed response curves, not dynamically fit predictions from the saved Gradient Boosting model in the API request path. The archive includes model-fitting scripts for curve parameters, but the runtime optimizer loads constants from PARAMS. Before treating the curve as a business forecast, the team should document its fitting data, calibration period, currency, and validation error.

6.7 Optimization Objective

For each candidate allocation, the optimizer evaluates the revenue curve for every selected platform and adds the estimates. SciPy's minimize function solves a minimization problem, so the code returns the negative of total predicted revenue. Minimizing this negative value is equivalent to maximizing the total modeled revenue.

```
def objective(budgets):
    total_revenue = sum(
        predicted_revenue(platform, budget)
        for platform, budget in zip(platforms, budgets)
    )
    return -total_revenue
```

This objective assumes platform revenues can be added independently. It does not model audience overlap, cross-channel interactions, attribution windows, saturation spillovers between platforms, or uncertainty. These assumptions define the scope of the optimization result and should be stated when interpreting the recommendation.

6.8 Constraints and Solver Call

The decision variables are one budget per selected platform. Each variable is bounded between that platform's minimum and maximum. A single equality constraint requires the variables to sum to the requested total. The code calls SLSQP, a constrained numerical optimization method available through SciPy.

```
result = minimize(
    objective, initial, method="SLSQP",
    bounds=list(zip(minimums, maximums)),
    constraints={
        "type": "eq",
        "fun": lambda x: np.sum(x) - total_budget
    }
)
if not result.success:
    raise RuntimeError(f"Optimization failed: {result.message}")
```

The result.success check prevents a failed numerical solve from being presented as a successful allocation. A more complete API should also return solver status, constraint residuals, and a structured error payload so that users can distinguish infeasible requests from numerical failures.

6.9 Feasibility Check

Before calling the solver, the module calculates the sum of minimum platform budgets and the sum of maximum platform budgets. If the requested total falls outside that interval, it raises a ValueError with the feasible lower or upper bound. This check is mathematically sufficient for these independent interval bounds and one total-equality constraint: any total between the sums can be formed by distributing the residual budget within the available platform capacities.

Feasibility condition	Meaning
total < sum(minimums)	Too little budget to meet all selected minimums
total > sum(maximums)	Too much budget for the selected maximums
otherwise	A bound-feasible allocation exists

A solver can still fail for numerical reasons even if the bounds imply feasibility, so the result status check remains useful. Boundary values should be included in future validation because rounding and floating-point tolerances can affect displayed totals.

6.10 Starting Point and Allocation Assembly

The optimizer starts from a feasible allocation. It initializes every platform at its minimum and distributes the remaining budget in platform order until the total is reached or the next platform reaches its maximum. SLSQP then adjusts that starting vector to improve the objective. The returned vector is paired with platform names to create the allocation dictionary.

```
initial                                   =                                   minimums.astype(float)
remaining                =                total_budget                -                minimums.sum()
for                         i                          in                          range(len(platforms)):
    addition          =          min(remaining,          maximums[i]          -          minimums[i])
    initial[i]                                  +=                                      addition
    remaining                                   -=                                      addition
    if                              remaining                           <=                          0:
        break
```

The order-based initial point is a starting condition, not the final allocation rule. The solver is responsible for exploring feasible alternatives. It is still good practice to test whether different initial points lead to materially different optima for the same request.

6.11 Gradient Boosting Training Pipeline

The training script reads the processed CSV, selects campaign context and planned budget as features, and uses revenue as the target. Categorical variables are one-hot encoded, while month, day-of-week, weekend flag, and planned budget pass through numerically. The preprocessing and estimator are wrapped in one scikit-learn Pipeline so that the same transformations are applied at fit and prediction time.

```
features = ["platform", "campaign_type", "industry", "country",
            "month", "day_of_week", "is_weekend", "planned_budget"]
target = "revenue"
y = np.log1p(df[target])
model = GradientBoostingRegressor(
    n_estimators=200, learning_rate=0.05, max_depth=3, random_state=42
)
```

The script uses a random 80/20 train-test split with a fixed seed. A time-ordered split is often more informative for campaign data because future performance is the intended prediction setting. The stored test metrics should therefore be read in light of the split design.

6.12 Target Transformation and Metrics

The model is fitted to log1p(revenue), a transformation that can reduce the effect of a long right tail. Before calculating MAE, RMSE, and R-squared, the code applies expm1 to both predictions and held-out targets to return them to the revenue scale. This means the reported errors are expressed in the target's original currency units, according to the dataset.

```
predicted_log                                  =                                pipeline.predict(X_test)
y_pred                                       =                                 np.expm1(predicted_log)
y_actual                                          =                                   np.expm1(y_test)
mae                          =                        mean_absolute_error(y_actual,                    y_pred)
rmse                       =                     np.sqrt(mean_squared_error(y_actual,                  y_pred))
r2 = r2_score(y_actual, y_pred)
```

The artifact metrics file reports MAE 17134.3413, RMSE 27743.9019, R-squared 0.21984, 1,440 training rows, and 360 test rows. The values should be reported with dataset currency and test protocol, neither of which is fully encoded in that small JSON file. These metrics do not describe the separate response-curve optimizer's performance.

6.13 Model and Optimizer Integration Gap

The training pipeline saves a fitted pipeline to campaign_revenue_model.pkl. However, backend/main.py imports optimize_budget from budget_response_optimizer.py, and that optimizer computes revenue through PARAMS and revenue_curve. No joblib.load call or prediction call to the saved Gradient Boosting artifact appears in this endpoint module. This source-level evidence means the currently archived API workflow does not connect the trained model to the optimizer.

This distinction resolves an ambiguity between the abstract and code. The abstract states the intended design: use a Gradient Boosting Regressor for prediction and use predicted outcomes for allocation. The archive shows two components that would need an integration layer to implement that end-to-end design. The report preserves the abstract as the project's stated aim and describes the code snapshot as it exists, without claiming the components are already connected.

A future integration should define one response interface, confirm feature availability for each candidate platform-budget pair, validate units, and compare its output with the existing fitted response curves before replacing them.

6.14 Data Preparation Script

The preprocessing script reads global_ads_performance_dataset.csv, reports basic data shape, column types, missing values, duplicates, platform counts, context categories, date span, and descriptive statistics. It also computes grouped summaries and spend quartiles for exploratory inspection. The raw file is read and the processed data are written separately, preserving the source CSV.

```
df_proc              =              df.drop(columns=["date_parsed",              "spend_quartile"]).copy()
df_proc["date"]                             =                              pd.to_datetime(df_proc["date"])
df_proc["year"]                                  =                                 df_proc["date"].dt.year
df_proc["month"]                                 =                                df_proc["date"].dt.month
df_proc["day_of_week"]                            =                             df_proc["date"].dt.dayofweek
df_proc["is_weekend"]             =             df_proc["day_of_week"].isin([5,            6]).astype(int)
df_proc["planned_budget"] = df_proc["ad_spend"]
```

The script derives calendar fields and aliases observed ad_spend to planned_budget. For retrospective model evaluation this can be a reasonable proxy, but at prediction time the value must represent a proposed budget known before the campaign runs. The project should explicitly document this operational meaning to prevent target leakage or feature mismatch.

6.15 Feature Timing and Leakage Review

The listed training features include platform, campaign type, industry, country, month, day of week, weekend flag, and planned budget. These are intended to be known before or at campaign planning time. The archive's processed data also contains observed campaign outcomes such as clicks, conversions, and revenue; the training script does not list those outcomes as model inputs. Excluding post-campaign outcomes is a positive design choice for forward prediction.

The key risk is the planned_budget alias. It is assigned from ad_spend in the processed dataset. If ad_spend is historical realized spend, the feature is a proxy for a future proposed budget rather than an independently recorded planned amount. The training/serving contract should therefore verify that the value supplied during prediction has the same definition, currency, scale, and timing as the training feature.

A leakage review should also check duplicate campaigns across random train/test partitions, repeated observations from the same campaign, and whether date fields allow near-identical records on both sides of the split.

6.16 React Frontend Request Path

The React application uses Axios to send the budget and market to the API. The archived source hardcodes http://127.0.0.1:8000/optimize for both the main optimization and what-if requests. That address targets a local machine and is not a deployment-safe backend URL for a remotely hosted website unless a proxy or runtime rewrite is configured separately.

```
const response = await axios.post(
  "http://127.0.0.1:8000/optimize",
  { total_budget: numericBudget, market: market }
);
setResult(response.data);
```

The form collects additional fields such as campaign name, brand, industry, campaign type, duration, and revenue goal. In this archived request path, only total_budget and market are sent to the backend. This means the returned optimizer allocation is not conditioned on those other form values in the code snapshot.

6.17 Client-Side Validation and State

Before sending a request, the frontend checks that the budget field is present and lies within the displayed market range. It sets a loading state during the request, clears prior errors, and shows an error message if the request fails. On a market change it clears old results, errors, and budget entries so that stale allocations do not remain attached to a new market.

Client-side validation improves usability but cannot replace server-side checks. The backend's Pydantic range and optimizer feasibility checks provide the server boundary. The archived frontend parses a validation message from the response where available and otherwise displays a generic retry message. A robust deployment should also handle network timeouts, HTTP status codes, and server unavailability without leaving a loading indicator active.

The code uses explicit loading flags for the main and what-if workflows. Finally blocks would make cleanup more resilient if unexpected exceptions occur outside the Axios catch path.

6.18 What If Analysis

The what-if feature sends a second optimization request with a changed budget and the current market. It stores that response separately from the baseline result and calculates a predicted-revenue difference by subtracting the baseline total from the alternative total. Because both requests use the same API contract, the scenario varies total budget and market but does not alter campaign context, audience, or creative.

A what-if comparison is a model scenario, not a causal estimate. It answers how the current function responds to a different total under its coded assumptions. To support a more complete comparison, a report should show baseline and alternative totals, per-platform allocations, revenue estimates, and the absolute and percentage change, along with the model version and constraints used.

The interface should require a successful baseline before showing a change value. The report's earlier screen observations noted this expected sequence; the code confirms that the revenue difference is only computed when both result objects exist.

6.19 CORS and Runtime Configuration

The FastAPI application allows cross-origin requests from localhost ports 5173 and 127.0.0.1:5173, the common local Vite development origins. It permits credentials, all methods, and all headers. The archive does not configure the deployed Vercel origin in the visible backend CORS list, and the frontend request URL is also localhost. This source snapshot alone does not establish how the currently deployed version is configured.

For a public deployment, origins should be configured through an environment-specific allowlist, with only the necessary methods and headers. The API URL should be supplied through a build-time or runtime environment value rather than hardcoded. These settings should be verified against the deployed version before claiming that the archive reproduces the live service.

The archive does not include a backend requirements file at the expected location. The report therefore identifies libraries from imports and project code, but does not assert a fully reproducible environment or exact dependency versions for the server.

6.20 Data and Model Artifacts

The archive includes raw and processed CSV files and a serialized model artifact. The report does not reproduce the datasets or pickle contents. The source uses a model metrics JSON with the model name and summary scores. Binary model files can be sensitive to library-version compatibility and should not be treated as self-describing documentation.

For a reproducible handoff, retain the data dictionary, dataset source and license, extraction date, currency, target definition, processing command, random seed, package versions, and artifact checksum. Store model artifacts separately from source code where practical, and document who may access them. Do not load pickle files from unknown sources because deserialization can execute code.

The metrics in this report are transcribed from the archive's JSON file, not recomputed during this document task. No claim is made here that the training script was rerun or that the model was independently validated.

CHAPTER 7 - EVALUATION AND OPERATIONAL DESIGN

7.1 End to End Request Trace

The archived request begins when a user enters a budget and chooses a market. The React handler validates the value, sends JSON to /optimize, and waits for a response. FastAPI parses the body into BudgetRequest, checks the market label, and calls optimize_budget. The optimizer selects platforms, loads fixed response parameters, checks feasible totals, solves the constrained problem, computes per-platform outputs, and returns a JSON object. React stores the response and renders allocation and revenue summaries.

PROJECT PIPELINE
1. Campaign form
2. Axios request
3. FastAPI validation
4. Market platform set
5. SLSQP allocation
6. Revenue response
7. Dashboard update

The archived endpoint passes budget and market only; the model-training pipeline is a separate code path in the archive.

The trained Gradient Boosting model is not part of this request trace in the supplied source. That fact should remain visible in diagrams and implementation descriptions until an integration is present and verified.

7.2 Constraint Example for India

For an India request, the solver uses two decision variables: Google Ads budget and Meta Ads budget. Each has a minimum of ₹5,000. Google has a maximum of ₹25,000 and Meta a maximum of ₹15,000. For a total of ₹25,000, feasible pairs must sum to ₹25,000 while keeping Google in [₹5,000, ₹25,000] and Meta in [₹5,000, ₹15,000]. The optimizer searches this line segment for the largest sum of the two coded response curves.

Variable	Lower bound	Upper bound	Budget equality
Google Ads	₹5,000	₹25,000	$x_1 + x_2 = ₹25,000$
Meta Ads	₹5,000	₹15,000	$x_1 + x_2 = ₹25,000$

This description establishes feasibility but does not state a particular solution. Exact allocation values depend on the constants, solver result, and runtime environment. Sample outputs recorded from the website are presented separately and are not substituted for a reproducible code run.

7.4 Boundary and Error Cases

The main cases to check are the smallest feasible total, largest feasible total, just-below and just-above feasible totals, invalid market labels, missing budget, nonnumeric input, and optimizer failure. At the market boundaries only one allocation may satisfy all platform bounds. Near a boundary, solver tolerances should be checked against the displayed equality.

Case	Expected behavior
India total = ₹10,000	Both platforms receive their minimums
India total = ₹40,000	Both platforms reach their maximums
India total < ₹10,000	Reject as infeasible
Global total > ₹60,000	Reject as infeasible
Unknown market	Return a clear client error

The current API's broad Pydantic range admits values such as ₹50,000 for India, but the optimizer rejects them after market bounds are known. Error handling should make the market-specific feasible range explicit to the user.

7.5 Model Evaluation Plan

The archive's metrics file reports MAE, RMSE, R-squared, and row counts for the Gradient Boosting model. MAE is the average absolute error in revenue units. RMSE gives larger errors more weight. R-squared compares residual variation with total target variation; a value around 0.22 indicates limited explained variation under the stored test procedure. These values are not evidence of high predictive accuracy.

Evaluation should use a time-aware holdout if dates represent campaign chronology, compare a simple baseline such as mean revenue or historical channel average, and report error by platform and budget range. It should include calibration plots or residual analysis and document any outlier handling. If multiple rows come from the same campaign, split by campaign identity to reduce leakage across train and test.

Model quality should be assessed separately from optimizer feasibility. Feasibility checks verify constraints; they do not show that the objective function predicts actual outcomes or improves business performance.

7.6 Optimization Validation Plan

Optimization validation should verify the sum constraint, all platform bounds, platform selection by market, and stable behavior for repeated runs with identical inputs. A brute-force grid over a coarse range can provide an independent reference for a small number of channels. The numerical result should be compared with equal split, minimum-first, and historical allocation baselines.

Sensitivity analysis should vary the response-curve coefficients and total budget to reveal whether a small parameter change causes a large allocation shift. Monotonicity and diminishing marginal return assumptions should be checked over the supported budget interval. A recommendation that depends strongly on unvalidated parameters should be labeled exploratory.

The archive contains optimizer validation and test scripts, but their presence does not establish that they pass. The metrics and observations in this report are sourced from files or prior UI inspection; the scripts were not run as part of preparing this document.

7.7 Test Inventory and Evidence Status

The archive includes scripts named for allocation validation, budget coverage, curve checks, platform checks, and optimizer validation. These names indicate intended checks. This document task did not execute those scripts, and their outputs are not included as evidence. A final technical appendix should attach a dated test record with environment, command, result, and exceptions.

Evidence type	What it supports	What it does not support
Source code	Implemented logic in the snapshot	Successful runtime behavior
Metrics JSON	Stored metric values and split counts	Independent metric reproduction
Website screenshot	Visible UI state at capture time	Backend code identity
Validation script	A check has been authored	The check passes

This evidence table separates static code review from execution and observed deployment behavior. Keeping that distinction makes the report suitable for academic review without overstating tests.

7.8 Deployment and Integration Checklist

Before presenting the archive as the deployed service, verify the deployed backend URL, CORS allowlist, frontend API base URL, and source revision. The archived frontend calls localhost and the archived backend allows only local development origins. A remote deployment needs an appropriate network path and origin configuration. The report does not infer these settings from a successful website page load.

- Set the API base URL through an environment-specific configuration value.
- Allow only the intended production frontend origin in CORS.
- Return structured 4xx errors for invalid market and infeasible totals.
- Log request identifiers, model or curve version, and solver status without storing unnecessary personal data.
- Document deployment revision and confirm a remote optimization request reaches the expected backend.

These items are implementation recommendations based on the supplied snapshot. They are not represented as completed features.

7.9 Security and Data Handling

Campaign budgets and performance data can be commercially sensitive. The supplied API example does not show authentication, authorization, rate limiting, or a persistent database. The small endpoint may be suitable for a demonstration, but any production use should define access control, transport security, request limits, data retention, and audit logging. Secrets should be supplied through deployment configuration and excluded from source archives.

The archive contains third-party dependencies and data files. A clean handoff should exclude virtual environments, node_modules, caches, temporary datasets, logs, and credentials. Dependency manifests should be retained so that the application can be rebuilt without packaging installed libraries. The user-provided ZIP was reviewed as data; embedded files were not executed.

The serialized pickle artifact should only be loaded from a trusted build pipeline. The model artifact should have an integrity record and compatible dependency specification.

7.10 Limitations of the Current Snapshot

The endpoint optimizes fixed platform response curves and does not use the archived Gradient Boosting model. The training script uses a random split and stored R-squared is approximately 0.22. The budget feature is derived from historical spend. The frontend sends only budget and market despite collecting additional campaign fields. The archived API URL and CORS origins are local-development values. The archive does not show a complete backend dependency manifest.

These observations are specific to the supplied code archive. They do not prove that the live deployment has identical behavior. The report retains prior website observations with their date and labels them as deployment observations. A source revision or build identifier is needed to reconcile deployed behavior with the ZIP snapshot.

The implementation should be described as a prototype that demonstrates constrained allocation and a separate supervised-model workflow, with a clearly identified integration gap. Further evaluation is needed before interpreting predictions as operational forecasts.

APPENDIX B - SELECTED SOURCE EXCERPTS

B.2 Revenue Curve and Parameters

Only the function and representative platform parameter shape are reproduced here; the complete constants table is explained in the surrounding chapter.

```
PARAMS                                                =                                                    {
  "Google           Ads":           {"alpha":           38833.0620,           "beta":           0.00015731,
                "gamma":                   343.0009,                 "min_budget":                 5000,
                "max_budget":                                                                      25000},
  #        Meta         Ads        and        TikTok        Ads        use        separate        constants
}

def               revenue_curve(budget,               alpha,               beta,               gamma):
    return alpha * np.log1p(beta * budget) + gamma
```

The constants have units only through the data and currency conventions used when they were fitted. The archive does not include a standalone provenance record for each parameter. This is why the report describes them as fixed coefficients in the snapshot.

B.3 Constrained Optimization Core

This excerpt shows the essential relationship between the objective, variable bounds, total equality, and solver status.

```
result                                          =                                          minimize(
    objective,                                initial,                                method="SLSQP",
    bounds=list(zip(minimums,                                                             maximums)),
    constraints={"type":                                                                        "eq",
                 "fun":          lambda          x:          np.sum(x)          -          total_budget}
)
if                                          not                                         result.success:
    raise RuntimeError(result.message)
```

The bounds are supplied as one lower and upper value per platform. The equality function returns zero when the allocated budgets add to total_budget. The result vector is used only after the solver signals success.

B.4 Training Pipeline Features

The training script places categorical encoding and numerical pass-through inside a scikit-learn ColumnTransformer, then combines it with the Gradient Boosting estimator in a Pipeline.

```
categorical_features = ["platform", "campaign_type",
    "industry",                                      "country"]
numeric_features = ["month", "day_of_week",
    "is_weekend",                                   "planned_budget"]
preprocessor = ColumnTransformer([
    ("categorical", OneHotEncoder(handle_unknown="ignore"),
     categorical_features),
    ("numeric", "passthrough", numeric_features)
])
```

OneHotEncoder(handle_unknown='ignore') allows an unseen category to avoid an immediate encoding exception. It does not guarantee that a prediction for an unseen campaign category is reliable; that remains a data-coverage issue.

APPENDIX C - API AND DATA CONTRACT NOTES

C.1 Example Request and Response Shape

```
Request
{             "total_budget":             25000,             "market":             "India"             }

Response                                          (shape                                                only)
{
  "total_budget":                                                                                        25000,
  "market":                                                                                             "India",
  "allocation":          {"Google          Ads":          0,          "Meta          Ads":          0},
  "predicted_revenue":         {"Google         Ads":         0,         "Meta         Ads":         0},
  "total_predicted_revenue":                                                                               0
}
```

Zeroes are placeholders to show the JSON shape; they are not a claimed optimizer output. A real response contains numerical values from the solver execution.

C.2 Training Data Fields Used

Role	Fields
Categorical inputs	platform, campaign_type, industry, country
Numeric inputs	month, day_of_week, is_weekend, planned_budget
Target	revenue, transformed with log1p for model fitting
Other processed fields	Date parts, spend, impressions, clicks, CTR, conversions and related dataset columns

The model uses only the first two input groups and predicts the target. Other columns may support exploration and processing but are not listed in the training feature vector shown in the script. Training and serving code should share a versioned schema to prevent mismatches.

C.3 Data Preparation and Reproduction

The preprocessing script expects a raw CSV in data/raw and writes the transformed table to data/processed. It derives calendar features and adds planned_budget from ad_spend. The training script expects the processed file at a relative path. These relative paths make the working directory part of the runtime contract.

A reproducible run record should identify the working directory, Python version, package versions, dataset checksum, source revision, command, output path, random seed, and training metrics. The archive bundles an environment but does not provide a clean dependency file at the expected backend path, so the exact rebuild procedure should be documented before another machine reproduces the model.

APPENDIX D - GLOSSARY AND FINAL TRACEABILITY

D.1 Glossary

Term	Meaning in this report
Allocation	Budget assigned to each selected advertising platform
Response curve	Function mapping platform budget to an estimated revenue value
Feasible allocation	Allocation satisfying total budget and per-platform bounds
SLSQP	Sequential Least Squares Programming numerical optimizer
MAE / RMSE	Revenue-scale prediction error measures
R-squared	Coefficient of determination on a stated evaluation split
Source snapshot	The particular backend ZIP reviewed for this edition

D.2 Final Traceability Note

This edition combines the report PDF supplied by the team, the project source archive, prior project materials retained in the workspace, and prior inspection notes for the deployed interface. The final abstract remains the authority for the intended project aim. The source archive is the authority for the code-specific statements in Chapters 6 and 7. Where those sources differ, the report states the difference rather than silently merging them.

The selected code excerpts are short and limited to the API contract, response function, constrained solver, preprocessing, and training workflow. Third-party implementation code and bundled dependencies are not reproduced.